

Московский политехнический университет
Факультет информационных технологий
Кафедра «СМАРТ-технологии»
Образовательная программа «Киберфизические системы»

Программирование алгоритмов систем управления
Лабораторная работа № 5
Реализация многопоточных вычислений

Задание

Разработать приложение для моделирования параллельных вычислений.
Провести эксперименты по режимам работы с разной приоритезацией работы потоков и при разной степени вычислительной нагрузки

Задачи

1. Создать консольное приложение с поддержкой многопоточности

Подготовить приложение с инициализацией 5 нитей (потоков) для вычислений

2. Создать функцию обработки данных в потоке

Реализовать метод, реализующий функцию вычислений для потока, в соответствии с индивидуальным заданием.

3. Реализовать создание потоков и настройку приоритетов

Реализовать запуск вычислений в потоках, а также в главном потоке, с передачей идентификаторов потоков (идентификатор главного потока – «0», остальных – номер по возрастанию в формате строки).

4. Реализовать запуск потоков и провести анализ результатов

Разместить код старта потоков. Выполнить приложение и изучить вывод в консоли приложения.

5. Провести эксперимент с изменением приоритетов потоков

Провести несколько запусков приложения, изменяя приоритеты потоков. Проанализировать влияние приоритета и приоритетов других потоков на вывод приложения.

6. Создать инструмент управления нагрузкой и ресурсами ПК и визуализации режимов работы потоков

Интегрировать в приложение средство управления ресурсами ПК. Модифицировать приложение с сохранением (выводом) идентификаторов потоков в отдельный текстовый файл или поток и вставкой временных меток 5-10 раз в секунду. Затем реализовать функцию загрузки и анализа накопленных данных (файла) с построением графиков работы потоков.

Замечания по выполнению задач

Цель лабораторной работы заключается в исследовании влияния приоритезации потоков на их быстродействие в разных условиях, в том числе в плане доступности вычислительных ресурсов.

Основной поток консольного приложения обычно выполняется на уровне Normal и в наименьшей степени влияет на подчиненные потоки, а также предоставляет потоконезависимый ввод-вывод через методы пространства имен System.Console. Также должен поддерживаться вывод идентификаторов потоков в общий файловый поток (текстовый) для возможности сохранения результатов и последующей обработки. Направление вывода должно выбираться (настраиваться) при старте приложения (работа с файлом реализуется при выполнении задачи 6, т.е. является дополнительной).

Потоки должны выводить свои «идентификаторы», фактически, переданную им при старте строковую величину как параметр. Например, в приложении в первый поток передается строка «1», которую тот будет использовать для индикации своей работы. В качестве идентификатора основного потока нужно использовать строку «0».

В приложении необходимо реализовать сохранение временных отметок старта потоков и их завершения.

Важно помнить, что системные часы имеют точность в 10 – 16 ms из-за ограничений системного таймера, поэтому нужно проводить по несколько экспериментов с одинаковыми настройками для усреднения оценок. Из столь низкой точности оценки времени также следует, что длительность блока вычислений, проводимых потоками должна быть значительно больше данного интервала и составлять не менее нескольких секунд. В противном случае разница между результатами потоков будет существенно зависеть от момента запуска и моментов переключений на планировщик задач.

Для оценивания точного времени, проходящего между стартом и остановкой потока рекомендуется использовать метод `int TickCount()`. Этот метод подсчитывает количество миллисекунд с момента старта операционной системы. Также для оценки времени можно использовать свойство `DateTime.Ticks`, которое возвращает количество 100-наносекундных интервалов, прошедших с 01.01.0001, 00:00. Для контроля времени также можно использовать класс `Stopwatch`.

Для проведения экспериментов необходимо разработать достаточно сложную вычислительную функцию, чтобы вычисления оказывали значительный вклад в загрузку работы вычислительной системы. Хорошим вариантом является циклически повторяемые операции вроде $x = \text{Math.Sin}(x)$, с числом повторов более десятков тысяч раз. После окончания некоторого количества повторов нужно выводить в консоль или файл номер потока, так чтобы общее количество выводов составляло 50-100 и более раз. После запуска приложения в консоли должны появляться чередующиеся номера потоков в те моменты, когда потоки получают процессорное время. Все потоки должны проводить одинаковые вычисления для обеспечения сравнимости результатов.

При выполнении анализа нужно провести эксперименты с разным отношением вычислительных операций и операций ввода вывода, а также с количеством доступных вычислительных ресурсов.

В ходе работы нужно составить план эксперимента и провести исследования, начав от установки минимального приоритета для всех потоков и постепенно доведя до ситуации с максимальным приоритетом для всех потоков.

Также приблизительно (по выводу в консоль) оценить равномерность работы потоков. То есть, как распределяется процессорное время (поочередно, блоками последовательно и т.п.).

При наличии в операционной системе возможности ограничить количество ядер (на процессорах AMD), выделяемых процессу, нужно попробовать использовать только одно ядро, затем два и так далее. Также попробовать по-разному нагружать вычислительную систему запуском дополнительных «тяжелых» приложений. Хорошим вариантом является запуск интернет-браузера с большим количеством вкладок.

Еще одним направлением эксперимента является изменение соотношения между вычислительными операциями и операциями ввода вывода (печати номера потока в консоль).

Проанализировать работу приложения в разных режимах, результаты исследования привести в отчете.

Основными направлениями анализа являются выяснение влияния установленных приоритетов на выделение процессорного времени и на характер передачи управления планировщиком задач.

Платформа .NET предлагает инструмент приоритетного закрепления процессоров (ядер) за выполняемым приложением. Закрепление можно реализовать с использованием инструментами пространства имен **Process**. Например, закрепление первого процессора (ядра) за приложением реализуется следующей инструкцией:

```
Process.GetCurrentProcess().ProcessorAffinity = (IntPtr)0x1;
```

Также для обеспечения «внешней» повышенной нагрузки на центральный процессор можно использовать модифицированную версию создаваемого приложения, запускающего большое количество потоков с приоритетами Highest. Такое приложение может быть полезным при работе на ПК с большим числом ядер.

Замечания по задаче визуализации

Для работы с файлами рекомендуется использовать файловые потоки.

На платформе .NET файловые потоки являются потокобезопасными объектами, что позволяет работать с ними без использования инструментов контроля доступа (мьютексов).

Создаваемые вычислительные потоки должны записывать в файловый поток свой идентификатор так же как в задачах 1-5 проводится вывод на консоль.

Параллельно работе вычислительных потоков в основном потоке приложения должна проводиться запись временной метки на регулярной основе (5-10 раз в секунду), отмечая равномерные временные интервалы работы приложения. В качестве временной метки может использоваться какой-нибудь символ, например, «#» (шарп).

При проведении анализа сохраненного файла необходимо подсчитать, сколько раз каждый вычислительный поток публикует свой идентификатор в текущий временной интервал, после чего нужно построить графики с указанием этих значений. Высота каждого графика будет примерно отражать, сколько процессорного времени получал каждый поток.

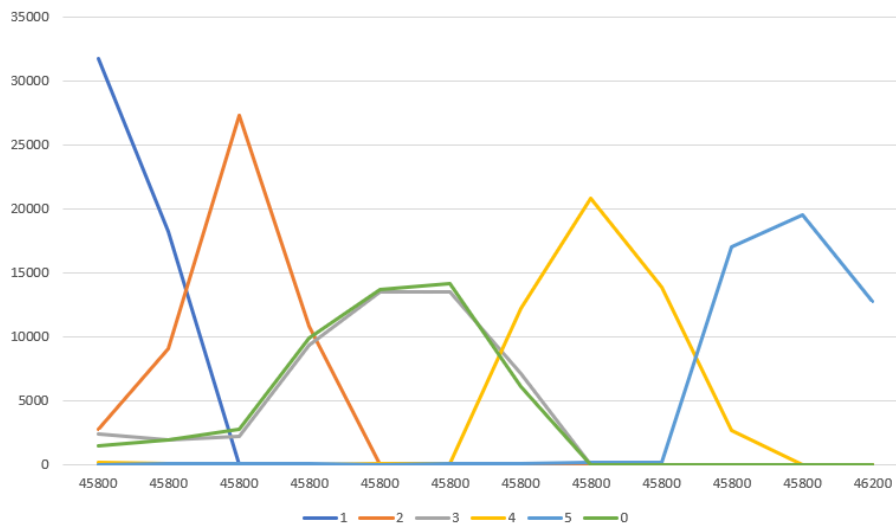


Рисунок 1 – Пример визуализации работы потоков с разными приоритетами при работе на одном выделенном ядре (поток 0 – основной поток приложения)

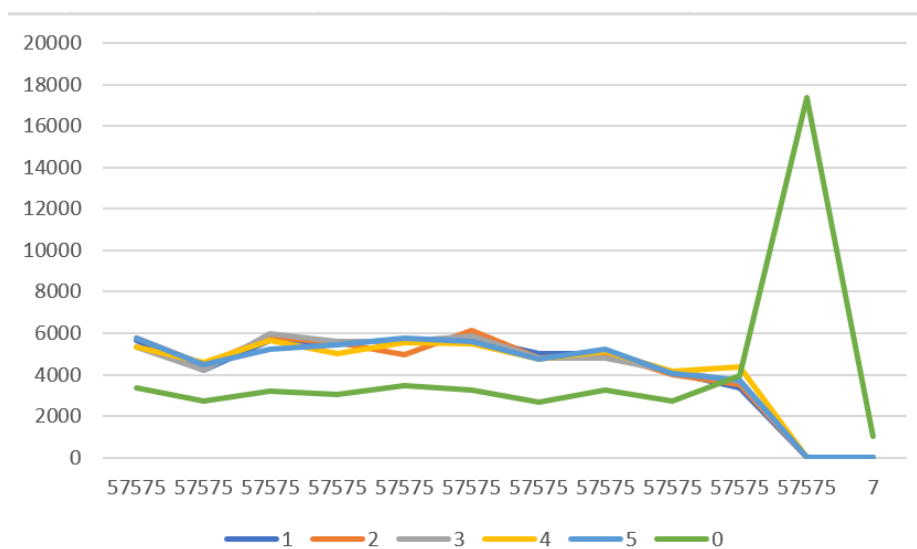


Рисунок 2 – Пример визуализации работы потоков с приоритетами Highest при работе на одном выделенном ядре (поток 0 – основной поток приложения)

Контрольные вопросы

1. Какое именно время будет подсчитано и выведено по завершению работы потока? В какой мере оно будет соответствовать реальному времени работы кода потока?
2. Какой вариант (стратегия) переключения потоков эффективен для высоконагруженных приложений (с большим объемом вычислений)? Какой вариант переключения потоков эффективен для сервисов, предназначенных для обслуживания запросов, например, веб-серверов?
3. Что произойдет с созданным приложением, если уменьшать или увеличивать долю вычислений по сравнению с выводом сообщений на консоль (или в файл) и почему? Как на это повлияет количество доступных вычислительных мощностей и ядер процессора?

Примеры по созданию и управлению потоками (нитеями)

// Разрешение пространств имен

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading;
```

// Статическая функция для реализации функциональности (вычислений) в потоках

```
static void WriteString(object _Data) {
    // для получения строки используем преобразование типов:
    // приводим переменную _Data к типу string и записываем
    // в переменную str_for_out
    string str_for_out = (string) _Data; // теперь поток 1 тысячу раз выведет полученную строку
    (свой номер)
    for (int i = 0; i <= 1000; i++)
    {
        for (int j = 0; j <= 1000; j++)
        {
            x = x + Math.Exp(x * 10); // нужно что-то долго считаемое,
                                     // но не выходящее за диапазон
                                     // допустимых значений

            x = 1 / x;
        }
        Console.Write(str_for_out);
    }
}
```

// Создание объекта-потока

```
Thread th_1 = new Thread(WriteString); // Упрощенный вариант, без использования делегата
                                         // ThreadStart
```

// Настройка приоритетов потоков

```
th_1.Priority = ThreadPriority.Highest; // .Highest - самый высокий;
                                         // .BelowNormal - выше среднего;
                                         // .Normal - средний;
                                         // .AboveNormal - ниже среднего;
                                         // .Lowest - низкий;
```

// Старт потока с передачей параметра

```
th_1.Start( "1");
```

// Ожидание завершения потока

```
th_1.Join();
```

// Ожидание нажатия на клавишу – полезно для консольных приложений, чтобы можно было увидеть результат

```
Console.ReadKey();
```

Замечания к учету времени:

Обратите внимание, что получать значение времени работы потока нужно в нем самом, а не в основном потоке. Полученные значения можно сохранить где-нибудь в общей переменной или в массиве. Например, использовать общую строковую переменную, в которую можно добавлять текст сообщения о прошедшем времени:

```
// Переменная для хранения сообщений
```

```
string resultmessage;
```

```
// Вывод сообщения, где worktime – это целочисленное значение времени в миллисекундах
```

```
resultmessage += "Поток " + str_for_out + " выполнил работу за " + worktime.ToString() + " ms /n";
```

```
// Итоговый вывод можно осуществить уже в основном потоке после остановки всех остальных потоков.
```

```
Console.Write(resultmessage);
```